PRACTICAL NUMBER: 8(iii)

Aim: Implement an Algorithm for detecting cycle in given graph G

iii. Accept graph as input in the form of adjacency matrix

Program: -

```
//SHEIKH WASIMUDDIN 60 23071044 CSEIOT//
#include <stdio.h>
#include <stdlib.h>

#define MAX_VERTICES 100

typedef struct Graph {
   int numVertices;
   int **adjMatrix;
   int *adjList[MAX_VERTICES];
} Graph;

Graph* createGraphFromUserInput() {
   Graph *graph = malloc(sizeof(Graph));

   printf("Enter the number of vertices: ");
   scanf("%d", &graph->numVertices);

   graph->adjMatrix = malloc(graph->numVertices * sizeof(int
*));
   for (int i = 0; i < graph->numVertices; i++) {
      graph->adjMatrix[i] = malloc(graph->numVertices *
sizeof(int));
   }

   printf("Enter the adjacency matrix (%d x %d):\n", graph-
>numVertices, graph->numVertices);
```

```
    for (int i = 0; i < graph->numVertices; i++) {
        for (int j = 0; j < graph->numVertices; j++) {
            scanf("%d", &graph->adjMatrix[i][j]);
        }
    }

    return graph;
}
int hasCycleUtil(Graph *graph, int v, int *visited, int *recStack) {
    if (!visited[v]) {
        visited[v] = 1;
        recStack[v] = 1;

        for (int i = 0; i < graph->numVertices; i++) {
            if (graph->adjMatrix[v][i]) {
                if (!visited[i] && hasCycleUtil(graph, i, visited, recStack))
{
                    return 1;
                } else if (recStack[i]) {
                    return 1;
                }
            }
        }
    }
    recStack[v] = 0;
    return 0;
}

int hasCycle(Graph *graph) {
    int *visited = calloc(graph->numVertices, sizeof(int));
    int *recStack = calloc(graph->numVertices, sizeof(int));

    for (int i = 0; i < graph->numVertices; i++) {
        if (hasCycleUtil(graph, i, visited, recStack)) {
            free(visited);
```

```
            free(recStack);
            return 1;
        }
    }
    free(visited);
    free(recStack);
    return 0;
}

void freeGraph(Graph *graph) {
    for (int i = 0; i < graph->numVertices; i++) {
        free(graph->adjMatrix[i]);
    }
    free(graph->adjMatrix);
    free(graph);
}

int main() {
    Graph *graph = createGraphFromUserInput();
    printf("Cycle Detected: %s\n", hasCycle(graph) ? "Yes" : "No");

    freeGraph(graph);
    return 0;
}
```

Output: -

main.c

```c
//SHEIKH WASIMUDDIN 60 23071044 CSEIOT//
#include <stdio.h>
#include <stdlib.h>

#define MAX_VERTICES 100

typedef struct Graph {
    int numVertices;
    int **adjMatrix;
    int *adjList[MAX_VERTICES];
} Graph;

Graph* createGraphFromUserInput() {
    Graph *graph = malloc(sizeof(Graph));

    printf("Enter the number of vertices: ");
    scanf("%d", &graph->numVertices);

    graph->adjMatrix = malloc(graph->numVertices * sizeof(int *));
    for (int i = 0; i < graph->numVertices; i++) {
        graph->adjMatrix[i] = malloc(graph->numVertices * sizeof(int));
    }

    printf("Enter the adjacency matrix (%d x %d):\n", graph->numVertices, graph->numVertic
    for (int i = 0; i < graph->numVertices; i++) {
        for (int j = 0; j < graph->numVertices; j++) {
            scanf("%d", &graph->adjMatrix[i][j]);
        }
    }

    return graph;
}
int hasCycleUtil(Graph *graph, int v, int *visited, int *recStack) {
    if (!visited[v]) {
```

```
Enter the number of vertices: 5
Enter the adjacency matrix (5 x 5):
0 1 1 0 1
0 1 1 0 1
0 0 1 1 1
1 0 0 0 1
0 0 0 1 0
Cycle Detected: Yes


...Program finished with exit code 0
Press ENTER to exit console.
```

Conclusion:

Hence, Implement an Algorithm for detecting cycle in given graph Gsuccessfully executed.